

04Categorizing-Data

October 5, 2021

1 Data Science: Bridging Principles and Practice

1.1 Part 4: Categorizing Rocket Fuel Data

1.1.1 Table of Contents

In this notebook, we will learn how to manipulate and visualize categorical data. We will then apply these tools to explore how consumer responses differ at different times or days in the Rocket Fuel case.

```
<li><a href="#section4">Categorizing Data</a>
  <ol type=a>
    <br>
    <li><a href="#section4a">Groupby</a></li>
    <br>
    <li><a href="#section4b">Visualization: Bar Plots</a></li>
    <br>
    <li><a href="#section4c">Consumer Response vs Day of the Week</a></li>
    <br>
    <li><a href="#section4d">Consumer Response vs Hour of the Day</a></li>
  </ol>
</li>
</ol>
```

```
In [1]: # load some necessary software: THIS CELL MUST BE RUN
import pandas as pd
import seaborn as sns
%matplotlib inline
import calendar
import warnings
import numpy as np
warnings.filterwarnings('ignore')
from gofer.ok import check
```

In the previous notebook, we learned some basic Python and Table operations and applied them to the Rocket Fuel data to answer questions about the profitability of the ad campaign.

Here, we'll build on those previous skills to dive deeper into the Rocket Fuel case. We'll explore questions including:

- How did conversions relate to different *hours of the day* and *days of the week*?

To answer these questions, we'll need to know a few additional Python operations .

2 4. Categorizing Rocket Fuel Data

As a reminder, our data look like this:

```
In [2]: # load the rocket fuel data
ads = pd.read_csv('data/rocketfuel_data_renamed.csv', index_col=0)

# change days to a categorical variable so it will sort correctly
cat_type = pd.api.types.CategoricalDtype(categories=list(calendar.day_name), ordered=True)
ads["most ads day"] = ads["most ads day"].astype(cat_type)

# display the first five rows
ads.head()
```

```
Out[2]:
```

	user id	test group	converted	total ads	most ads day	most ads hour
0	1069124	ad	False	130	Monday	20
1	1119715	ad	False	93	Tuesday	22
2	1144181	ad	False	21	Tuesday	18
3	1435133	ad	False	355	Tuesday	10
4	1015700	ad	False	276	Friday	14

This Table shows the conversion behavior of each user in the study. But, we're not interested in individual user behavior right now- we want to know the behavior of *all users in a specific category*, like everyone who saw the most ads on a Monday, or everyone who saw 200 ads in total.

In this section, we'll talk about: 1. The definition of **categorical data** 2. How to use DataFrame methods to **group data into categories** 3. Using **visualizations** to view and compare categories

Let's say we have data about some of the handbags TaskBella sells, including the color, price, and rating for each collected from different stores.

```
In [3]: # create the example handbag data Table
handbags = pd.DataFrame(data={"color":["black", "red", "red", "brown", "black"],
                              "price":[115.99, 130, 124.95, 144.99, 120.05],
                              "rating":[4, 4, 5, 3, 4]})

handbags
```

```
Out[3]:
```

	color	price	rating
0	black	115.99	4
1	red	130.00	4
2	red	124.95	5
3	brown	144.99	3
4	black	120.05	4

2.0.1 Data Types: Numerical and Categorical

This table has two different types of data: **numerical** and **categorical**.

Price and rating are **numerical**: they have numbers for values, and we can order those values along a scale from least to most.

Color is **categorical**: it has strings (text) for values, and those values can't really be ordered from least to most.

The type of data affects the kind of analysis we can do, in addition to how we visualize it. For now, we're going to focus on categorical data.

2.1 4a. Groupby

A question we might ask is how price or rating is different for different categories of color. To conduct this analysis, we want to do something like this:

1. Find all the possible colors
2. Sort all the rows of the DataFrame into groups, one for each unique color
3. Return a new DataFrame with one row for each color and information about that color of bag

To use groupby, call it on a DataFrame using dot notation and specify which column you want to group on as the argument.

```
In [4]: # group by handbag color
        handbags.groupby("color")
```

```
Out[4]: <pandas.core.groupby.groupby.DataFrameGroupBy object at 0x7feafeea3f28>
```

The output above looks pretty strange. After we've told the computer to group the data by color, it doesn't know what to do with the groups. Should it count the items in the group? Should it take the average?

So, when we group items in a DataFrame, we also must say how we want to *aggregate* the groups by specifying an *aggregation function*. For this example, let's get the counts of each color of bag.

```
In [5]: # group by handbag color and return the count of each
        color_counts = handbags.groupby("color").count()
        color_counts
```

```
Out[5]:
```

	price	rating
color		
black	2	2
brown	1	1
red	2	2

We now have a new DataFrame with the count of each number of handbag. Notice that the "price" and "rating" columns are the same because counts don't depend on price or rating.

EXERCISE: Use `group` to group the handbags by `"rating"`. Aggregate them using the `count` function.

```
In [6]: # group by rating
        rating_counts = handbags.groupby("rating").count()
        rating_counts
```

```
Out[6]:
```

	color	price
rating		
3	1	1
4	3	3
5	1	1

```
In [7]: # run this cell to check your answer
        check("tests/04/ratings.ok")
```

```
Out[7]: <gofer.ok.OKTestsResult at 0x7feb52a6dc50>
```

There are several aggregation methods besides count.

For example, if we want to know the average price and average rating of each color of handbag, we can use the mean method.

```
In [8]: # get average price and rating per color
        bag_avgs = handbags.groupby("color").mean()
        bag_avgs
```

```
Out[8]:
```

	price	rating
color		
black	118.020	4.0
brown	144.990	3.0
red	127.475	4.5

A picture might help to understand what just happened.

EXERCISE: We want to look at how conversion rates are different for different days of the week in the Rocket Fuel case. Fill in the ellipses below to group the ads data by the day on which a user saw the most ads, then add mean as the aggregation function.

Note: there's an extra argument in groupby called `as_index` which determines whether or not the groups will be the new DataFrame index. In this case, we don't want the groups as the index, so leave it set to `False`.

```
In [9]: # fill in the ... with the correct code
        day_rates = ads.groupby("most ads day", as_index=False).mean()
        day_rates
```

```
Out[9]:
```

	most ads day	user id	converted	total ads	most ads hour
0	Monday	1.318831e+06	0.032812	25.328517	14.608179
1	Tuesday	1.321255e+06	0.029840	23.925464	14.038191
2	Wednesday	1.317974e+06	0.024942	24.539205	14.287438
3	Thursday	1.310194e+06	0.021571	23.428852	14.511026
4	Friday	1.310578e+06	0.022212	26.612129	14.694454
5	Saturday	1.296354e+06	0.021051	25.227663	14.699963
6	Sunday	1.300229e+06	0.024476	24.403661	14.384197

```
In [10]: # run this cell to check your answer
          check("tests/04/day-avgs.ok")
```

```
Out[10]: <gofer.ok.OKTestsResult at 0x7feb52a82240>
```

Collection functions and data types:

The collect function you use must also work on the type of data in your rows for each group. For example, if you try to reduce using mean to get the average value and one or more columns has text data, those columns will be ignored since the computer doesn't know how to take the average of a word. You can see above that the text data in the "test group" column has been dropped.

However, Python treats Boolean values (True and False) like numbers: 1 for True and 0 for False. So, taking the average of the True/False values in the "converted" column is like adding up the number of True (people who converted) and dividing it by the total number of people in the DataFrame. This is the *conversion rate*: the proportion of people who bought a handbag after seeing an ad or psa.

The conversion rate will be key to our questions about whether the advertising campaign was effective.

2.2 4b. Visualization: Bar Plots

DataFrames provide great ways to organize and display data. But as data sets grow very large (i.e. thousands, tens of thousands, even millions of rows), it becomes harder and harder to understand what's going on with the data just by looking at it in a table.

Visualizations are helpful to: - get a big-picture understanding of a data set - compare two or more variables - find the variance of a variable and much more.

Let's look again at the DataFrame we just made containing the average feature values for each day.

```
In [11]: day_rates
```

```
Out[11]:
```

	most ads day	user id	converted	total ads	most ads hour
0	Monday	1.318831e+06	0.032812	25.328517	14.608179
1	Tuesday	1.321255e+06	0.029840	23.925464	14.038191
2	Wednesday	1.317974e+06	0.024942	24.539205	14.287438
3	Thursday	1.310194e+06	0.021571	23.428852	14.511026
4	Friday	1.310578e+06	0.022212	26.612129	14.694454
5	Saturday	1.296354e+06	0.021051	25.227663	14.699963
6	Sunday	1.300229e+06	0.024476	24.403661	14.384197

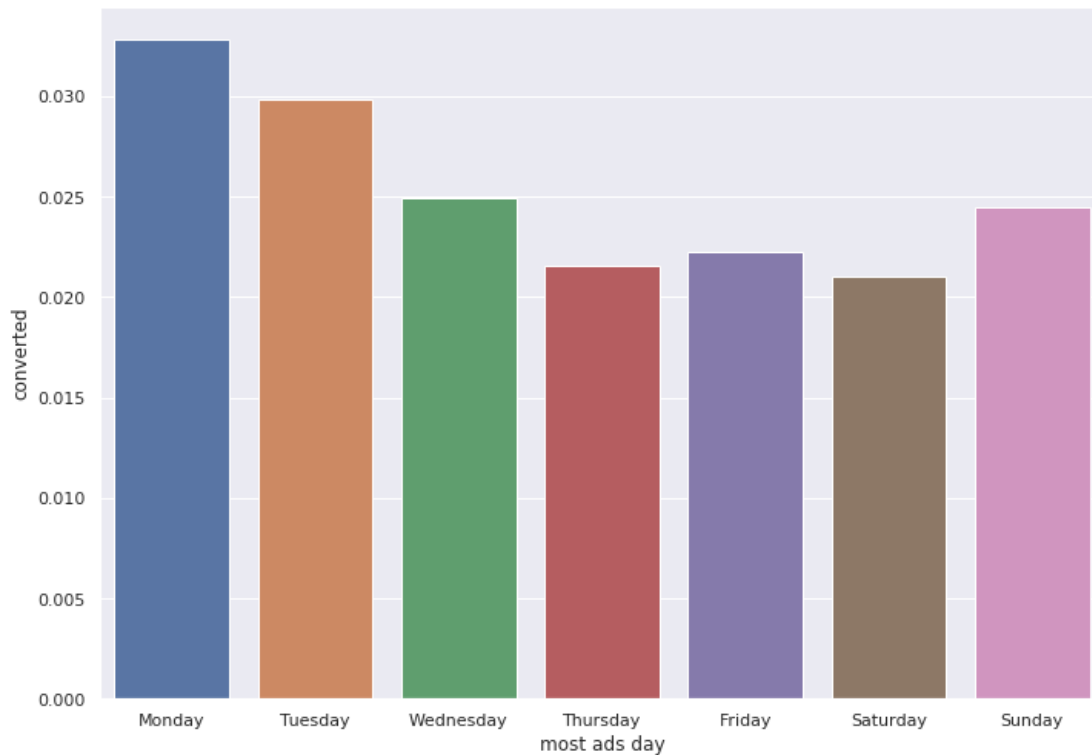
One of the best ways to visualize categorical data is with a **bar plot**. Bar plots allow us to compare multiple categories within the same plot.

To make our plots, we'll be using a software library called **Seaborn**. Seaborn is built to make polished visualizations in Python without a lot of code. The standard abbreviation for Seaborn in Python code is `sns`, after [a character in the TV show The West Wing](#).

We will make a bar plot by calling the `barplot` method from the Seaborn (`sns`) library. `barplot` takes 3 arguments: 1. `x`: the name of the column with the categories that will go on the x-axis 2. `y`: the name of the column with the numerical data that will determine the height of the bars on the y-axis 3. `data`: the name of the DataFrame

```
In [12]: # make the plots bigger
sns.set(rc={'figure.figsize':(11.7,8.27)})

# plot a barplot of the conversion rates for each day
sns.barplot(x="most ads day", y="converted", data=day_rates);
```



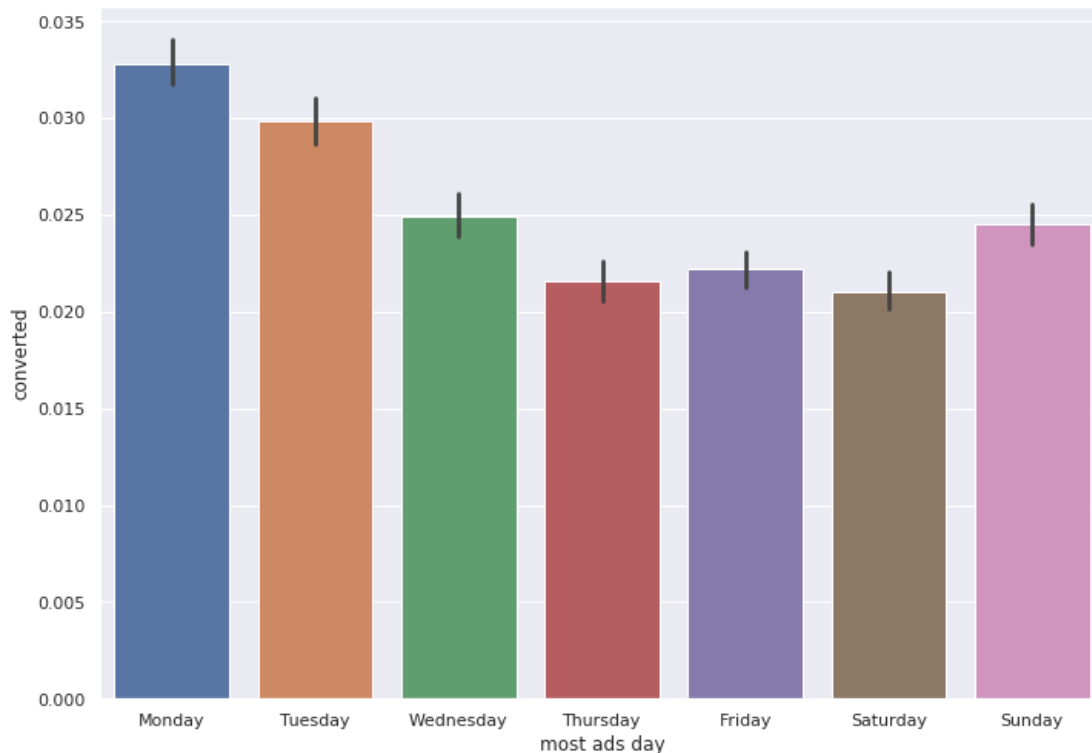
Each category is listed on the horizontal axis and represented by a bar. The length of each bar is the conversion rate.

Note: the expressions that create our bar plots are followed by a semicolon (;). This tells Python not to print out extraneous info about the result of the call expression.

If you want, you can try deleting the semicolon and re-running the cell to see why the semicolon is there.

It's also good to know that Seaborn will do grouping and averaging automatically. If we call `barplot` on our ungrouped ads table and ask it to plot "most ads day" on the x-axis and "converted" on the y-axis, it will group the rows by day and find the average of the "converted" column (i.e. the conversion rate) behind-the-scenes.

```
In [13]: # make a barplot with the original ads data (ungrouped)
sns.barplot(x="most ads day", y="converted", data=ads);
```



Notice that we've generated almost the same plot; the only difference is that when the data is ungrouped, Seaborn adds error bars.

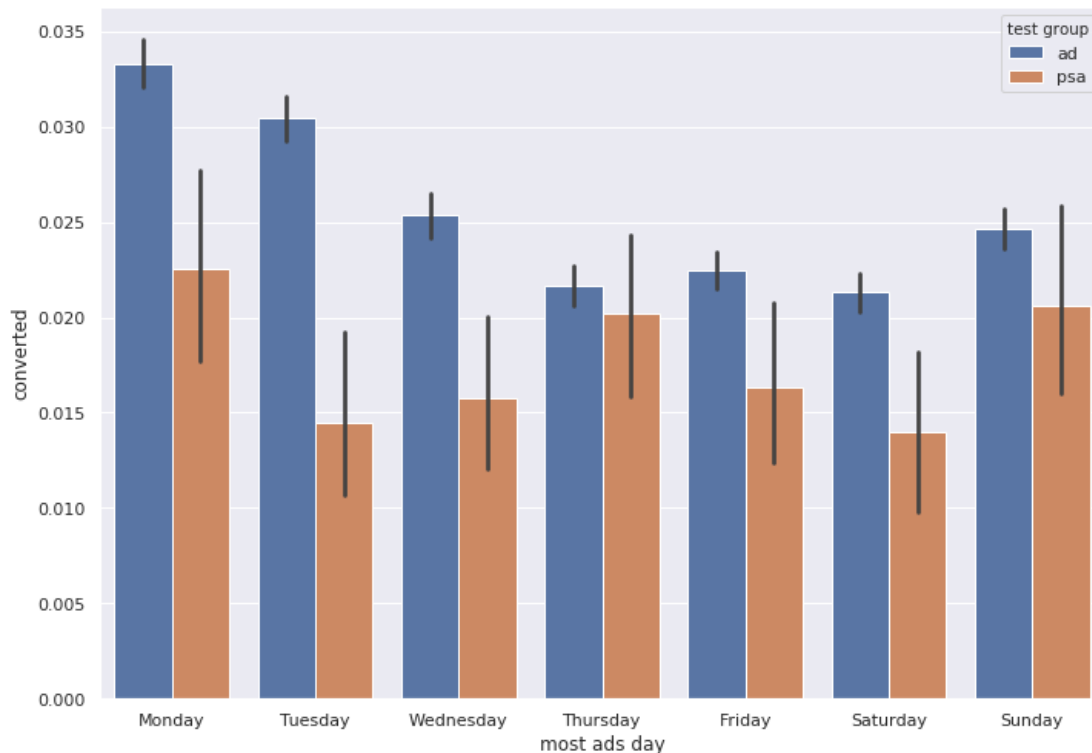
2.3 4c. Consumer Response vs. Day of Week

Suppose we want to return to our original question: how does conversion rate differ between different days of the week *for the two different test groups*?

Seaborn can handle this by adding an extra argument to `barplot`. The `hue` argument takes the name of a categorical column in the data, and it will make a different hue of bars for each category.

Here, we've chosen `test_group` to set the hue so that bars representing the "ads" group will be blue and bars representing the "psa" group will be orange.

```
In [14]: # make a bar plot with different colors for each test group
sns.barplot(x="most ads day", y="converted", hue="test_group", data=ads);
```



QUESTION: On which days is advertising the most effective? When is it least effective?

ANSWER: Fill in your answer here

Looking at the test group that saw the ads Monday is most effective and least effective is Thursday and Saturday

2.4 4d. Consumer Response vs Hour of Day

Next, we want to see how the conversion rate does or does not change for each test group by *time of day*.

EXERCISE: Create a bar plot showing the consumer response vs the hour of the day.

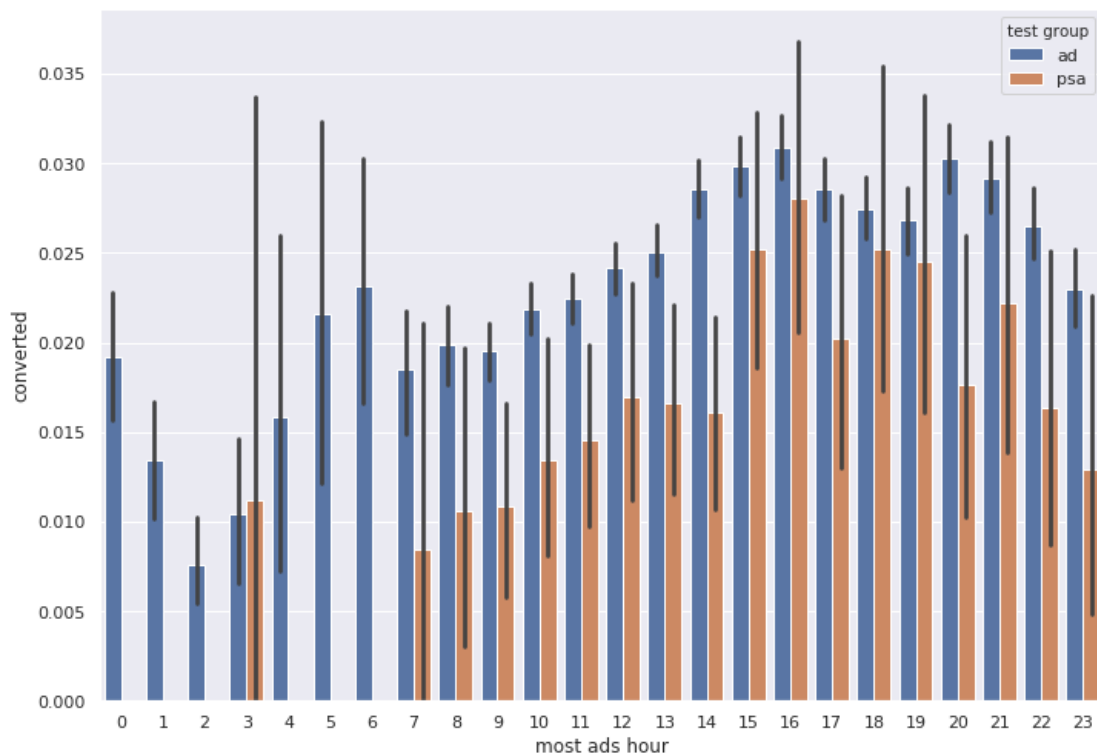
- The categories (`x`) should be the different hours of the day
- The height of the bars (`y`) should be the conversion rate
- The hue should be the test groups
- The data should be from the `ads` DataFrame

Look at the "Conversion Rate vs Day of Week and Test Group" example for some hints- you should

As a reminder, here's the data dictionary for our ads DataFrame:

user id	test group	converted	total ads	most ads day	most ads hour
The unique identifier for that user	Which testing group the user was in: "ad"- where users saw the ads (the experimental group) or "psa"- where users saw the PSAs (the control)	Whether or not the user bought a hand-bag (True if they did, False if they didn't)	The total number of ads (or PSAs) seen by that user	The day of the week on which the user saw the most ads (or PSAs)	The hour of the day during which the user saw the most ads (or PSAs)

```
In [15]: # make a bar plot
sns.barplot(x="most ads hour", y="converted", hue="test group", data=ads);
```



QUESTION: At which hours is advertising the most effective? When is it least effective?
</div>

ANSWER: Fill in your answer here Looking at the test group that saw the ads Hour 15 is most effective and least effective is Hour 2

References

- Rocket Fuel data and discussion questions adapted from materials by Zsolt Katona and Brian Bell, BerkeleyHaas Case Series

Author: Keeley Takimoto